Design and Deployment of an FPGA-Ready Distributed Market Data Research Testbed

A Senior Project Report

presented to
the Faculty of California Polytechnic State University
San Luis Obispo

In Partial Fulfillment

of the Requirements for the Degree

Bachelor of Science in Computer Engineering

By

David El Kaim

CPE 462

Advisor: Dr. Maria Pantoja

Winter 2025

Abstract 2

This project designed and deployed a heterogeneous, FPGA-ready research testbed for controlled market-data processing experiments. The system combines a custom AMD Ryzen Linux workstation, an NVIDIA Jetson Orin Nano edge node, and a Mac mini control and observability node. A dedicated wired laboratory subnet provides an experimental data plane, while Tailscale supplies secure remote administration across the three computers. Prometheus, Node Exporter, and Grafana provide persistent infrastructure monitoring and make processor, memory, storage, network, and node-availability metrics visible from a centralized dashboard. The software reference path generates synthetic FIX-style quote events, transmits the feed over TCP, reconstructs and validates complete messages, writes normalized records for KDB+/Q ingestion, and derives rolling top-of-book and bid/ask quantity analytics. The C++ implementation also reports sender-to-receiver latency percentiles, and its standalone verification suite completed 33 tests with no failures. Development followed a dependency-driven progression: hardware provisioning, separated control and data networking, persistent observability, a reproducible software reference path, source-level validation, and finally a defined migration boundary for programmable logic. No RTL was completed during the documented period; however, the resulting event format, expected outputs, host platform, tests, and measurements place a bounded RTL parser or analytics module as the next active engineering stage.

Table of Contents

1. Introduction

Electronic trading systems operate as pipelines. Market events arrive from a feed, are decoded and normalized, update an internal representation of market state, trigger analytics or decisions, and are written to storage for later investigation. A conventional backtest often begins after several of these steps have already been abstracted away. It may operate on bars or cleaned historical records rather than on the ordered event stream that an online system would actually receive. As a result, the research code can be separated from the networking, timing, data integrity, operating system, and observability problems that determine whether a real system is dependable.

The purpose of this senior project was to build the physical and software environment required to study those problems in a controlled laboratory. The project began with deterministic FPGA processing as the acceleration objective, but an accelerator cannot be evaluated in

isolation. It first requires a repeatable feed source, explicit framing rules, known-correct outputs, a host interface, a storage path, synchronized measurements, and visibility into the surrounding computers. Development therefore proceeded in dependency order. The workstation and network established the physical platform, observability made the platform diagnosable, and the C++ and KDB+/Q path established an executable reference model. This sequence produced a useful system at every stage and reduced the RTL step from an open-ended integration problem to a bounded migration task.

The completed system consists of three heterogeneous nodes. The B650 Linux workstation is the primary development and feed-generation node. The NVIDIA Jetson receives the TCP stream, writes validated events to a CSV bridge, and can run the q aggregation process. The 2018 Mac mini acts as a persistent control plane and hosts Prometheus and Grafana. The systems communicate through a dedicated physical network for experiments and through a Tailscale overlay for secure administration. The software reference path generates FIX-style messages, sends them through TCP, validates checksums, fields, and sequence continuity, stores typed events in KDB+/Q through a file-ingest bridge, computes a rolling best-bid/best-ask view, and calculates normalized bid/ask quantity imbalance.

1.1 Motivation

The central motivation is that infrastructure quality affects the validity of systems research. A signal or model cannot be evaluated independently of the way data reaches it. Missing sequence numbers, stale state, queueing, process scheduling, network delay, storage delay, and poor observability can all change the behavior of a real time application. The project does not claim to reproduce a production exchange or institutional trading platform. Instead, it creates an institutionally inspired laboratory in which important components can be studied separately and then integrated.

An FPGA was selected as the next acceleration target because programmable logic can implement fixed streaming data paths with explicit cycle-level behavior. AMD describes Zynq devices as combining Arm processors with programmable logic, while Alveo cards provide PCI Express-attached acceleration for data-center workloads [10], [11]. A credible hardware result requires more than an RTL module: it requires stable inputs, expected outputs, repeatable tests, a defined insertion point, and a software comparison baseline. The completed platform establishes those conditions so that the first hardware module can be evaluated as part of an end-to-end system rather than as an isolated simulation.

1.2 Stakeholders

The project has three primary stakeholder groups:

- The student developer and future researchers, who need a reproducible environment for studying event driven market data systems, latency, and heterogeneous computing.
- The senior project advisor and Computer Engineering program, which require evidence of system design, engineering decisions, implementation, testing, and reflection.
- A future systems or quantitative infrastructure engineer, who would use the platform to compare software and hardware implementations under controlled workloads.

These stakeholders informed the design in different ways. Academic evaluation favored traceable requirements, diagrams, a bill of materials, and honest testing. Research use favored modularity, replayability, and accessible tooling. Future FPGA work favored an x86 Linux host with available PCI Express resources, a high capacity power supply, fast local storage, and a network interface that would not immediately constrain later experiments.

1.3 Project Goal and Objectives

The overall goal was to design and deploy a distributed market data research testbed that can support a software reference pipeline and later FPGA acceleration.

The measurable project objectives were:

1. Assemble and configure at least three networked compute nodes with clearly separated roles.
2. Provide a reliable control plane for remote administration without exposing laboratory services directly to the public Internet.
3. Create a dedicated local data plane for repeatable node to node experiments.

4. Deploy persistent centralized observability using Prometheus, Node Exporter, and Grafana.
5. Implement a synthetic, sequenced, timestamped market data path from generation through network reception and KDB+/Q storage.
6. Reconstruct basic top-of-book state and compute at least one real time market microstructure metric.
7. Instrument the pipeline so that latency distributions can be computed.
8. Define a stable event interface, expected outputs, insertion point, and host platform for the first FPGA processing module.
9. Validate encoding, decoding, framing, malformed-input handling, sequence behavior, and output formatting with automated tests.

1.4 Deliverables and Outcomes

The main deliverables produced by the project are:

- A custom Linux workstation prepared for high speed network and future FPGA accelerator cards.
- A three node heterogeneous laboratory connected by a physical switch and a Tailscale control network.
- Persistent node monitoring and visualization through Prometheus and Grafana.
- A configurable C++ FIX-style sender and TCP receiver with stream framing, checksum, field, and sequence validation.
- A seven-field CSV bridge and q aggregation script that populate typed KDB+/Q quotes and rolling book tables.
- Sender-to-receiver latency instrumentation that reports rolling P50, P95, P99, and maximum delay.
- A self-contained C++17 verification suite that completed 33 tests with no failures.
- A public source repository and runbook that preserve the software baseline for reproducibility and RTL test-vector generation.
- Architecture diagrams, engineering requirements, test procedures, and a bill of materials.

The most important outcome is a functioning research platform that makes networking, storage, process health, and market-data state visible. Just as importantly, the project produced a stable hardware/software boundary. The event format, framing behavior, reference outputs, automated tests, host resources, and measurement path are now defined, allowing RTL development to proceed as the next modular stage without redesigning the surrounding system.

2. Background

2.1 Electronic Trading Messages and Market State

The Financial Information eXchange protocol is a family of messaging specifications used for electronic trade communications [2]. Production market data systems may use exchange specific binary protocols, FIX variants, or internal normalized formats. This project uses a simplified FIX-style representation because it is readable, familiar, and sufficient for exercising framing, sequence validation, timestamps, network transport, and downstream storage. The generator is a research workload rather than an exchange certified FIX engine.

A market data receiver must preserve event order and maintain consistent state. Sequence numbers help identify missing, duplicated, or out of order messages. Timestamps allow delay to be separated into source, network, processing, and ingestion stages. At the simplest level, the receiver can maintain a top-of-book view consisting of the best bid price and size and the best ask price and size. A full Level 2 or Level 3 book would require multiple price levels or individual order identifiers and queue priority. Those features were outside the completed scope and are reserved for future work. Limit order book research emphasizes that realistic order flow and state modeling remain nontrivial even when high quality data is available [13].

2.2 Replay and Reproducibility

A replay system delivers a stored event stream in a defined order and timing relationship. Reproducibility is important because a pipeline cannot be compared across changes if every test uses a different event path. The testbed therefore treats sequence numbers, timestamps, and append only storage as first class design elements. A deterministic synthetic generator can also create controlled bid, ask, and volume skews that make analytics easier to validate. The platform is intended to hide future events until their scheduled replay time, reducing one common form of look ahead error in research workflows.

2.3 KDB+/Q for Time Series Data

KDB+ is a high performance time series, column oriented database with an in memory compute engine and the q programming language [3]. Its data model is well suited to timestamped events, and the KX documentation describes its use as a real time streaming processor and historical time series database. For this project, KDB+/Q provides an append only event store, compact typed tables, symbol and time filtering, and a natural location for top-of-book and derived analytics. It also creates continuity with real market data infrastructure practices without requiring a large distributed database deployment.

2.4 Observability

Observability is necessary because a multi node system can fail even when each program appears correct in isolation. Prometheus stores labeled time series and collects metrics by scraping configured HTTP endpoints [4]. Node Exporter exposes operating system and hardware metrics such as processor, memory, storage, and network statistics [5]. Grafana queries data sources such as Prometheus and presents the results as dashboards [6]. In this project, the Mac mini runs the persistent monitoring stack so that the primary workstation can be rebooted, reconfigured, or loaded without removing visibility into the rest of the laboratory.

2.5 Network Separation and Remote Administration

The design uses two logical network planes. The experimental data plane is a local wired subnet, 10.10.10.0/24, connected through a physical switch. The control plane uses Tailscale for encrypted point to point connectivity and remote access. Tailscale uses WireGuard based encrypted tunnels between nodes [7]. Separating the planes reduces dependence on ordinary WiFi or household routing for experiments and also provides a reliable fallback path for administration when a local interface is misconfigured.

2.6 FPGA Acceleration and Staged Migration

The target architecture places an FPGA at the boundary between network ingress and the software storage layer. Candidate functions include feed parsing, message normalization, sequence checks, timestamp capture, book-state updates, and imbalance calculations. The Dell Mellanox ConnectX-4 interface and the workstation's power, cooling, and PCI Express budget were selected with this migration path in mind. NVIDIA documents ConnectX-4 as a PCI Express server-adapter family for low-latency data-center and high-performance-computing connectivity [8]. An AMD Alveo U50 was considered because AMD positions the card for financial computing and provides PCI Express, HBM, and high-speed connectivity [11]. The Vitis and Vivado environments support simulation, synthesis, and implementation flows for FPGA development [12]. Development was staged so that the hardware phase would begin only after the message contract, reference behavior, tests, host platform, and observability path were stable. The C++ and q implementation now serves as the golden functional baseline for that migration. No Verilog or SystemVerilog source, synthesis result, bitstream, or physical FPGA processing is claimed for the documented period.

3. Formal Project Definition

3.1 Customer Requirements

Because this was an individual research project, the primary customer was the student researcher, with the advisor acting as the academic stakeholder. The customer requirements were:

- The platform must be affordable enough to assemble from a combination of new, used, and already owned components.
- The system must support remote development and monitoring across macOS, x86 Linux, and ARM Linux.
- The data path must be reproducible and observable rather than a single opaque application.
- The platform must support timestamped and sequenced market events and a KDB+/Q storage layer.
- The architecture must remain extensible to a future FPGA without requiring the entire laboratory to be rebuilt.

- The implementation must remain modular so that one processing stage can be replaced by RTL without redesigning the source, storage, or monitoring layers.
- The project must clearly distinguish implemented features from proposed high performance features.

3.2 End User Persona and Use Case

The representative end user is a computer engineering student or junior infrastructure researcher who understands basic Linux and networking but does not have access to an exchange colocation facility. The user wants to generate a repeatable event stream, send it through multiple computers, inspect storage and analytics results, and observe resource usage. The user values transparency, low cost, and modularity more than production scale.

Primary use case:

1. The user connects to the laboratory through the Tailscale control plane.
2. The user verifies that the B650 workstation, Jetson, and Mac mini are reachable and visible in Grafana.
3. The user starts or replays a synthetic market data feed on the source node.
4. The receiver reconstructs messages, validates sequence numbers, and forwards normalized records to KDB+/Q.
5. The user queries events and derived top-of-book or imbalance data.
6. The user examines system and pipeline metrics to identify bottlenecks or failures.
7. In the next development phase, the user substitutes an FPGA stage for one bounded software function and compares its outputs and timing with the reference implementation.

3.3 Engineering Requirements

Spec.	Parameter	Requirement or Target	Tolerance	Risk	Compliance
1	Compute nodes	At least 3 networked nodes	Min.	L	I, T
2	Network separation	At least 2 logical planes: control and data	Min.	M	I, T
3	Remote administration	All 3 nodes reachable without public port forwarding	Req.	M	T
4	Node metrics	CPU, memory, storage, and network metrics from at least 3 nodes	Min.	M	T
5	Dashboard visibility	Display current metrics for at least 3 nodes	Min.	L	I, T
6	Message integrity	Detect any sequence gap of 1 or more messages	Min.	M	A, T
7	Event persistence	Append all valid messages in the functional test to KDB+/Q	100%	M	T
8	Market state	Refresh rolling best bid, best ask, and associated quantities	1 s max	M	A, T

Spec.	Parameter	Requirement or Target	Tolerance	Risk	Compliance
9	Imbalance range	Normalized output remains between -1 and +1 for nonzero total quantity	[-1, 1]	L	A, T
10	Latency instrumentation	Capture source/receive timestamps and report P50, P95, P99, and maximum delay	Req.	M	I, T
11	Service persistence	Monitoring services return after host reboot without reinstallation	Req.	M	T
12	RTL migration readiness	Define FPGA insertion point, event fields, and reference outputs for one RTL stage	Req.	M	A, I

Table 1. Engineering requirements for the distributed market data research testbed.

Compliance codes: A = analysis, T = test, I = inspection, and S = similarity to an existing design.

3.4 Primary Constraints

The project was constrained by cost, hardware availability, time, and integration complexity. High-speed FPGA accelerator cards and compatible network equipment can cost more than the rest of a student laboratory. Used server adapters reduce acquisition cost but introduce firmware, cable, optics, and driver uncertainty. The B650 desktop platform provides modern compute and storage but has fewer PCI Express lanes and slots than an enterprise server. The RTX 2070, high-speed NIC, and next-stage FPGA card compete for physical space and host resources. Cross-architecture deployment also required different binaries and service behavior on x86 Linux, ARM Linux, and macOS.

The project also had a strict dependency chain. An FPGA stage required a stable feed source, explicit framing, known reference outputs, timestamping, a host interface, and a way to diagnose failures. The work was therefore organized as a sequence of atomic layers rather than parallel speculative features. Completing the infrastructure, observability, and software reference path reduced uncertainty at the eventual hardware boundary. Full-depth books, matching, strategy, and execution simulation remained outside the defined scope so that the platform could reach a coherent transition point into RTL.

4. System Design

4.1 Overall Architecture

Figure 1 shows the physical and logical architecture. Each node was assigned a distinct role instead of duplicating the same software on every machine. The B650 workstation performs the primary generation, development, and high-capacity compute work. The Jetson provides the ARM receiving and edge-processing node. The Mac mini remains powered as the control and observability server. The dashed FPGA block marks the next-stage insertion boundary: it is positioned where a verified software function can be replaced while the source, storage, and monitoring layers remain unchanged.

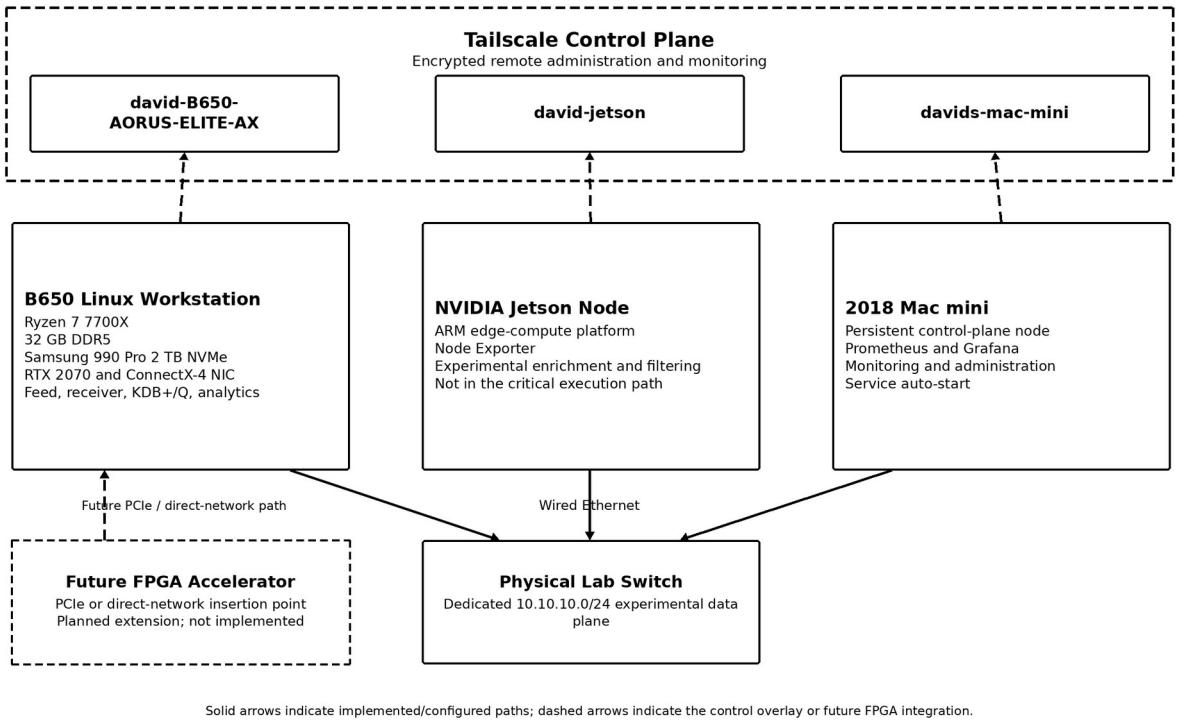

Figure 1. Physical topology, network planes, and future FPGA insertion point.

4.2 B650 Linux Workstation

The primary workstation was custom assembled around an AMD Ryzen 7 7700X processor and a Gigabyte B650 AORUS ELITE AX motherboard. The system includes 32 GB of Corsair Vengeance DDR5 memory rated at 6400 MT/s, a 2 TB Samsung 990 Pro NVMe solid-state drive

rated up to 7450 MB/s, an NVIDIA RTX 2070 GPU, an NZXT Kraken Plus 240 liquid cooler, a Corsair RM1200e power supply, and an NZXT H9 Flow chassis. A Dell Mellanox ConnectX-4 CX455A QSFP28 network interface provides a high-speed expansion path. BIOS bring-up verified the motherboard, Ryzen 7 7700X, 32 GB memory capacity, and Samsung 990 Pro 2 TB drive. At the photographed bring-up checkpoint, the memory operated at approximately 4800 MT/s with the EXPO profile disabled.

This node was selected for four reasons. First, the Ryzen platform provides sufficient general purpose performance for generation, decoding, KDB+/Q queries, and development tools. Second, the NVMe drive provides capacity and high sequential performance for event files and databases. Third, the large case and 1200 W power supply provide physical and electrical margin for accelerator cards. Fourth, Linux offers direct access to networking tools, systemd services, KDB+/Q, and AMD FPGA development software. The workstation is therefore both the present software pipeline host and the future FPGA host.

4.3 NVIDIA Jetson Edge Node

The NVIDIA Jetson Orin Nano provides a low-power ARM Linux platform and serves as the receiving and edge-compute node. It was integrated into Tailscale, the local laboratory network, and the Prometheus monitoring system. In the completed software path, the Jetson receives and validates the TCP feed before producing the CSV stream consumed by q. Its heterogeneous ARM environment also remains available for later event enrichment, filtering, or lightweight inference experiments.

The Jetson also created a useful integration challenge. Node Exporter initially encountered unsupported or inefficient hardware collectors. The systemd service was adjusted to disable collectors such as hwmon, thermal_zone, and zfs where they caused problems. After the service change, the metrics endpoint became available for Prometheus scraping. This debugging work

demonstrated that observability software must be adapted to the hardware and kernel capabilities of each platform.

4.4 Mac mini Control and Observability Node

The monitoring node is a 2018 Intel Mac mini with a 3.6 GHz quad core Intel Core i3 processor, 8 GB of memory, and a 128 GB internal solid state drive. It hosts Prometheus and Grafana and remains separate from the primary experimental workload. This separation improves visibility during B650 workstation reboots or software failures. The Mac mini also provides a convenient local browser endpoint for dashboards and a stable Tailscale address for remote access.

4.5 Network Architecture

The physical laboratory subnet uses the private range 10.10.10.0/24. Host aliases used during setup included fpga-lab, jetson-lab, and mac-mini-lab. The wired subnet is intended for experimental traffic. Tailscale hostnames included david-B650-AORUS-ELITE-AX, david-jetson, and davids-mac-mini. The Tailscale network is the reliable management path for SSH, service access, and monitoring when the physical subnet is unavailable or being reconfigured.

At one checkpoint, Prometheus targets addressed through the physical lab aliases returned a no route to host error while the Tailscale control path remained available. This was an important design result rather than merely a failure. It showed why control and data planes should be independent. A routing or interface problem on the experimental path should not remove the ability to inspect and repair the machines. The final report does not claim that the laboratory sustained 100 Gb/s traffic. The ConnectX 4 card provides interface capability and a future path, but an end to end 100 GbE benchmark was not completed.

4.6 Observability Architecture

Figure 2 illustrates the monitoring path. Node Exporter instances expose host metrics, Prometheus scrapes and stores those metrics, and Grafana queries Prometheus to construct

dashboards. The services were configured for persistent startup so that monitoring survived ordinary reboots. This converted the lab from a collection of computers into a system with centralized health information. The live Grafana dashboard is shown in the accompanying demonstration video rather than duplicated as a static screenshot in this report.

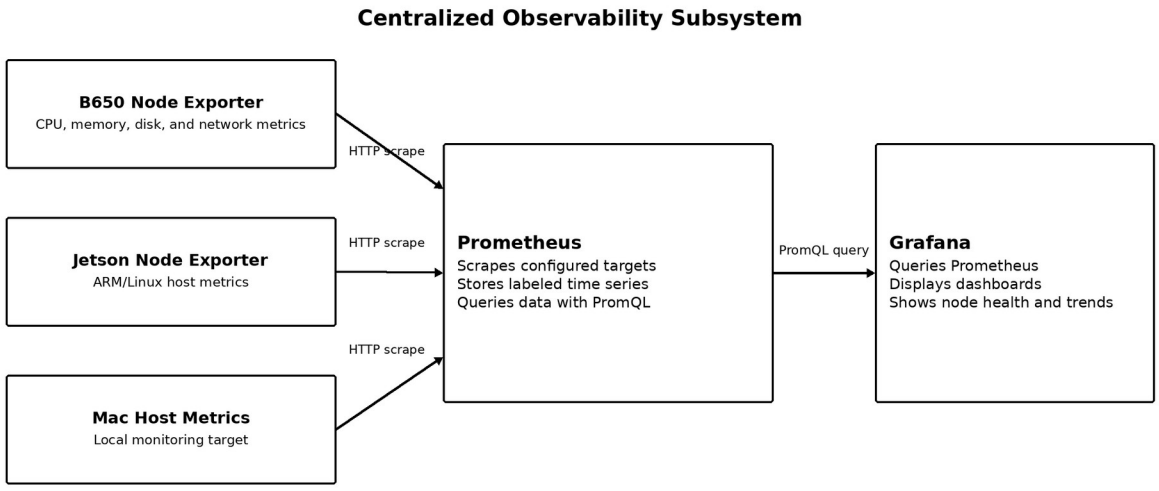

Figure 2. Centralized Prometheus and Grafana observability path.

4.7 Staged Development Progression

The platform was developed as a stack of independently testable layers. Each stage produced a usable artifact, reduced integration uncertainty, and exposed a stable interface to the stage above it.

- Phase 1 - Hardware foundation: assemble the B650 workstation, provision storage, power, cooling, and PCI Express expansion capacity.
- Phase 2 - Connectivity: establish secure remote administration and a separate wired subnet for experimental traffic.
- Phase 3 - Observability: deploy Node Exporter, Prometheus, and Grafana so failures remain visible while the data path changes.

- Phase 4 - Software reference path: implement a deterministic sender, receiver, framing contract, validation rules, KDB+/Q schema, and rolling analytics.
- Phase 5 - Verification: exercise protocol behavior with 33 automated tests and preserve the implementation in a reproducible repository and runbook.
- Phase 6 - RTL migration: replace one bounded software stage with a cycle-defined hardware implementation and compare it against the established reference under identical input.

This progression makes RTL the immediate continuation of the existing architecture rather than a separate redesign. The first hardware milestone can focus on a fixed-width parser or imbalance unit because the surrounding data source, expected behavior, and measurement environment already exist.

5. Implementation

5.1 Physical Assembly and Platform Bring Up

The workstation assembly required component selection, physical installation, power planning, storage installation, cooling installation, and expansion-card planning. The NZXT H9 Flow chassis was selected for airflow and card clearance. The NZXT Kraken Plus 240 liquid cooler supported CPU thermal management, while the Corsair RM1200e provided substantial power margin for the Ryzen processor, GPU, high-speed NIC, and a future accelerator. The Samsung 990 Pro provided a single high-performance local data volume. The ConnectX-4 adapter was obtained used, which lowered cost but required attention to the exact Dell part number, QSFP form factor, driver support, and compatible cabling. Figure 3 documents the primary workstation during assembly, the installed ConnectX-4 adapter, and BIOS-level hardware verification.

The three nodes were then joined into the same administrative environment. Each system required host naming, network interface configuration, Tailscale enrollment, remote access validation, and monitoring service installation. Cross platform differences made this work more substantial than copying a configuration file between identical Linux servers. The Mac mini used

macOS service management, while the Linux nodes used systemd. The Jetson required ARM compatible packages and collector changes.

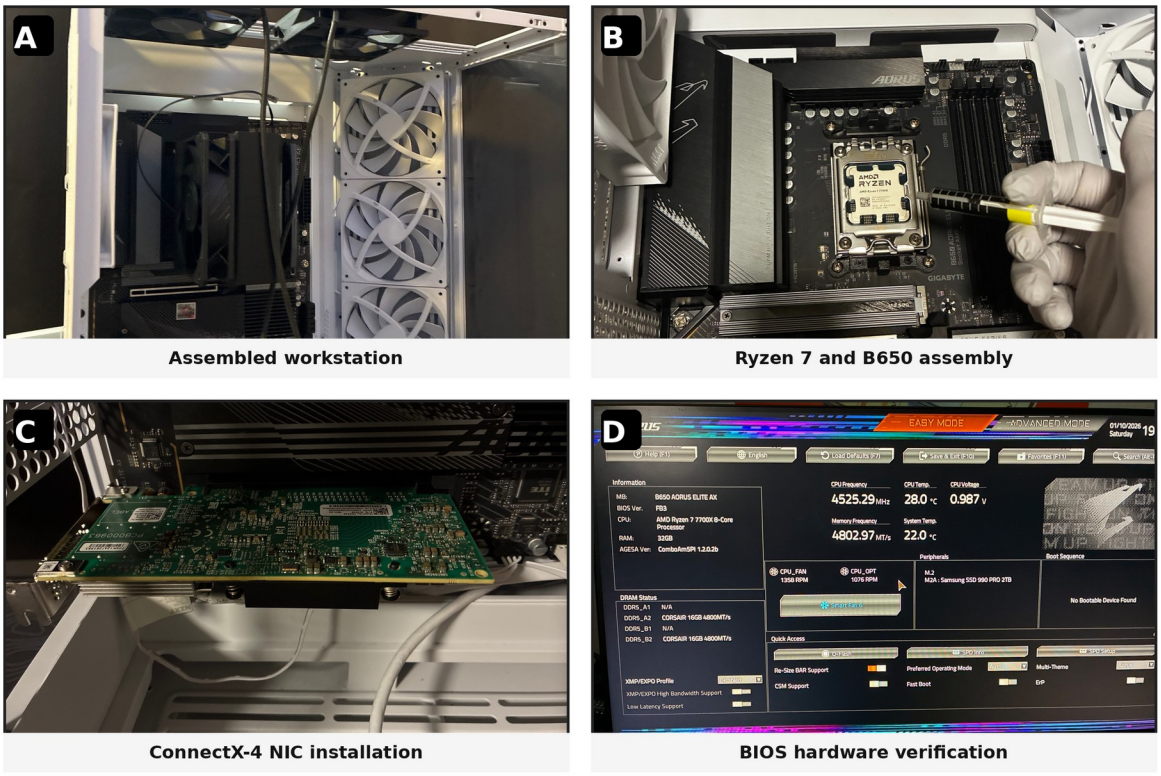

Figure 3. Hardware evidence from the primary workstation: (A) integrated workstation chassis, (B) Ryzen 7 7700X and B650 motherboard assembly, (C) Dell Mellanox ConnectX-4 network adapter installation, and (D) BIOS verification of the motherboard, processor, memory capacity, and Samsung 990 Pro 2 TB drive.

5.2 Market Data Software Path

Figure 4 shows the implemented software reference path. The C++ sender creates timestamped and sequenced quote events for NVDA, TSLA, AMD, PLTR, and COIN. It transmits newline-delimited FIX-style frames over TCP port 9009 with TCP_NODELAY enabled. The C++ receiver handles stream framing, checksum and field validation, sequence monitoring, CSV output, and rolling latency percentiles. The q process tails inbox.csv every second, deduplicates sequence numbers, appends typed rows to the quotes table, and derives a five-second rolling book table. TCP was selected for the first complete prototype because it simplified reliable delivery while the project stabilized framing, validation, storage, and observability. The

components are intentionally separated so that the receiver's decode or analytics stage can later be replaced without changing the generator or downstream data model.

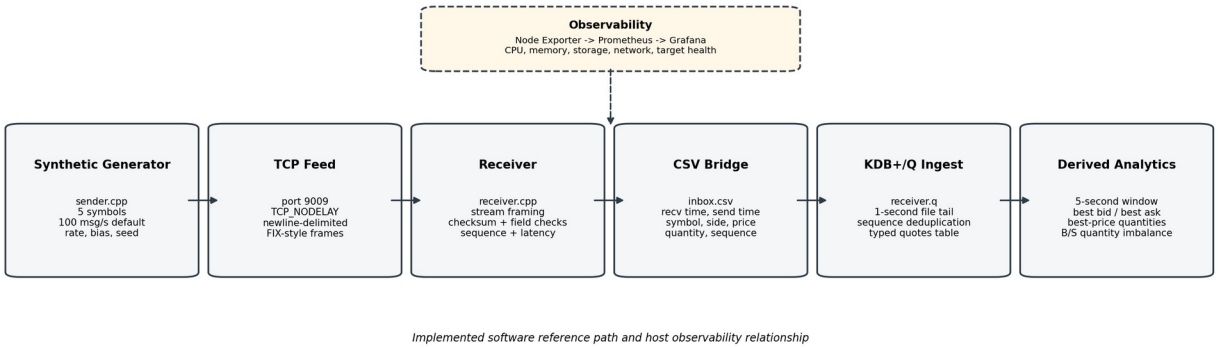

Figure 4. Source-verified software market data generation, reception, CSV bridging, KDB+/Q ingestion, analytics, and observability path.

5.3 Synthetic Feed Generation

The generator produces a controlled stream rather than relying on a live market connection. At its default setting it targets 100 messages per second, although the rate is configurable from the command line. A fixed seed can reproduce the pseudo-random symbol, price-step, side, and quantity sequence. Each selected symbol follows a small random walk with a maximum step of approximately 0.05 percent of its current price per generated event. The default side bias produces B-side events 70 percent of the time. B-side quantities range from 50 to 500, while S-side quantities range from 10 to 200. Every message includes a sequence number and CLOCK_REALTIME nanosecond timestamp.

The messages use a simplified FIX 4.4-style tag-value representation with SOH field delimiters and a newline frame delimiter. Standard tags are used where practical, while B/S side values and nanosecond timestamp fields are intentionally simplified for the research workload. The generator supports configurable host, port, rate, side bias, and random seed parameters. It is not presented as an exchange-certified FIX session implementation.

5.4 TCP Receiver, Framing, and Validation

TCP provides a byte stream rather than preserving application message boundaries. The receiver therefore accumulates bytes, extracts newline-delimited frames, retains partial data between reads, and processes multiple complete messages when they arrive together. It verifies the FIX checksum, accepts only PLTR, TSLA, AMD, NVDA, and COIN, requires B or S as the side, and rejects nonpositive prices or quantities. Sequence numbers are compared with the preceding accepted message, and discontinuities are counted and reported. Accepted records are written to inbox.csv using the columns recv_ts_ns, send_ts_ns, sym, side, px, qty, and seq.

The implemented latency measurement uses the sender timestamp and the receiver arrival timestamp. The receiver computes recv_ts_ns minus send_ts_ns and reports P50, P95, P99, and maximum latency over a rolling five-second sample window. Because CLOCK_REALTIME is used on separate hosts, meaningful cross-host values depend on clock synchronization. Decode-completion and KDB-ingestion timestamps are not present in the recovered source files, so the report does not claim separate decode or database-stage latency measurements.

5.5 KDB+/Q Event Storage

The q script defines an in-memory quotes table with the exact fields recv_ts_ns, send_ts_ns, sym, side, px, qty, and seq. Every second, a timer reads newly appended CSV lines, parses them into typed values, rejects malformed rows, deduplicates by sequence number, and inserts unseen records. The final recovered source corrects the incremental file offset so that the first newly appended row in each timer cycle is not skipped. The script also exposes an insQuote function for a possible later direct IPC path, but the completed prototype uses the file-ingest bridge.

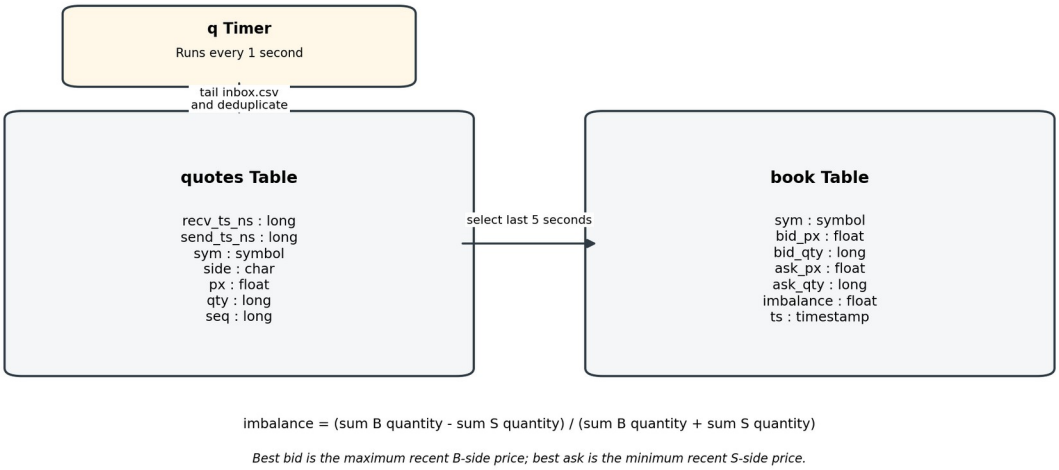

Figure 5. KDB+/Q event schema, one-second ingest timer, and derived rolling book table implemented in receiver.q.

5.6 Rolling Top-of-Book View and Bid/Ask Quantity Imbalance

The q analytics layer selects events received during the preceding five seconds. For each symbol, it defines the best bid as the maximum recent B-side price and the best ask as the minimum recent S-side price. The reported bid and ask quantities are the most recent quantities observed at those best prices. This is a rolling research view rather than a persistent Level 2 or Level 3 order book.

$$I = (Q_B - Q_S) / (Q_B + Q_S)$$

Here Q_B is the sum of B-side quantities and Q_S is the sum of S-side quantities in the same five-second window. A positive value indicates greater recent B-side quantity, a negative value indicates greater recent S-side quantity, and zero indicates balance. The ratio remains between -1 and +1 when the denominator is nonzero. Because the metric aggregates generated quote-side quantities rather than executed trades, it is described as rolling bid/ask quantity imbalance rather than trade order-flow imbalance.

5.7 Monitoring Deployment and Persistence

Prometheus was installed on the Mac mini and configured to scrape Node Exporter endpoints. Grafana was connected to Prometheus and made accessible through the private Tailscale network. The monitoring services were configured to run after reboot. The dashboard

provided a single view of the laboratory and helped identify unavailable targets, resource usage, and routing problems. This subsystem is also a prerequisite for the hardware phase: FPGA timing data is not meaningful if host saturation, routing faults, or service failures cannot be distinguished from accelerator behavior.

5.8 Source Repository and Reproducibility

The project source is publicly archived at https://github.com/DavidEK123/KDB_Q_FIX_observability_pipeline [16]. The repository separates the CMake build configuration, C++ sender, C++ receiver, q analytics, standalone verification suite, deployment runbook, and plain-language architecture explanation. The build and run instructions preserve the two-node workstation-to-Jetson flow rather than only a local demonstration.

This repository is part of the engineering output. It provides an executable software reference model, records the message and CSV interfaces, and gives the next RTL stage a known source of deterministic inputs and expected outputs. The same tests can be converted into hardware testbench vectors as functions migrate from software to programmable logic.

6. System Testing and Analysis

6.1 Test Strategy

Testing was divided into infrastructure, protocol, data integrity, storage, analytics, and performance categories. The C++ test program is self-contained and exercises the FIX encoder and decoder, checksum behavior, TCP-style frame splitting and coalescing, field validation, sequence anomalies, latency arithmetic, and CSV formatting. It compiled under C++17 and reported 33 passed tests and 0 failed tests. Integration tests were also performed as services were installed and network paths changed. Beyond validating the current software, these cases establish the first set of golden behaviors to preserve during RTL migration.

ID	Test	Procedure	Observed Result	Status
T1	Node reachability	Ping or connect to all three nodes through Tailscale	All three nodes reachable	Pass
T2	Local data plane	Connect nodes through the physical switch and private subnet	Interfaces configured; one routing issue observed during monitoring	Partial
T3	Metrics endpoint	Request Node Exporter metrics from each configured node	Metrics returned after Jetson collector fix	Pass
T4	Prometheus targets	Inspect target health and query up metric	Control-plane scraping operational; lab aliases exposed routing issue	Pass / Partial
T5	Grafana dashboard	Open dashboard remotely and inspect host metrics	Dashboard accessible through Tailscale	Pass
T6	Service persistence	Reboot or restart monitoring host and inspect services	Services configured for automatic startup	Pass
T7	Feed transport	Start generator and receiver on separate nodes	Messages delivered over TCP	Pass
T8	C++ protocol unit suite	Compile and run test_fix.cpp	33 of 33 tests passed	Pass
T9	q ingest source	Inspect schema, timer, parser, and deduplication	Typed quotes and book tables implemented in source	Pass by inspection
T10	Rolling analytics	Inspect five-second q selections and formula	Best recent B/S prices and normalized quantity ratio implemented	Pass
T11	Latency percentile logic	Inspect receiver aggregation	Percentile logic verified; final cross-host benchmark not archived	Partial
T12	FPGA execution	Process feed in programmable logic	No RTL implementation completed	Not performed

Table 2. Functional and integration test summary.

6.2 Infrastructure Results

The three node control network was successfully established, and the Mac mini could host a persistent monitoring service independent of the primary workstation. Prometheus queries confirmed which targets were available, while Grafana provided a readable view of system activity. The Jetson Node Exporter problem was resolved by changing the collector configuration rather than abandoning the node. This result demonstrates that the laboratory is maintainable across heterogeneous operating environments.

The local experimental subnet was useful but not completely transparent. A no route to host error appeared when Prometheus attempted to scrape selected lab aliases. The existence of the Tailscale control plane prevented this issue from blocking access to the machines. The incident supports the original architectural decision to separate administrative reachability from the network under test.

6.3 Market Data Pipeline Results

The software implements the complete reference path at source level: synthetic quotes are encoded by sender.cpp, transmitted over TCP, framed and validated by receiver.cpp, written to inbox.csv, and consumed by receiver.q. The q script defines both the typed quotes table and the five-second rolling book table. The standalone C++17 verification suite completed 33 tests with no failures. A final runtime KDB+/Q screenshot was not preserved, so the report distinguishes source-verified behavior and automated tests from missing presentation evidence. The verified path nevertheless defines the functional contract that the first RTL module must reproduce.

The project does not claim complete market microstructure fidelity. Individual order identifiers, queue priority, full depth, matching rules, slippage, fees, and market impact were not established as completed features. The top-of-book and imbalance analytics should be interpreted as controlled research signals, not as proof of profitable trading performance.

6.4 Latency and Throughput Analysis

The receiver directly implements source-to-arrival latency measurement using nanosecond CLOCK_REALTIME timestamps and reports P50, P95, P99, and maximum delay over a rolling five-second window. This verifies the percentile aggregation mechanism. A controlled final cross-host benchmark with documented synchronization status, hardware configuration, message rate, duration, and archived output was not recovered. Numeric performance values are therefore not reported as final testbed results.

Implemented evidence: receiver.cpp maintains a rolling five-second latency sample window and prints P50, P95, P99, and maximum sender-to-receiver delay. Final cross-host numerical benchmark output was not recovered, so no hardware-performance values are claimed.

Table 3. Latency instrumentation and final benchmark status.

6.5 Requirements Compliance Matrix

Spec.	Requirement	Status	Evidence or Note
1	At least three nodes	Pass	B650 workstation, Jetson, and Mac mini configured.
2	Two logical network planes	Pass	Physical 10.10.10.0/24 data plane and Tailscale control plane documented.
3	Remote administration	Pass	Nodes reachable through private Tailscale hostnames.

Spec.	Requirement	Status	Evidence or Note
4	Metrics from at least three nodes	Pass	Node Exporter and Prometheus were configured across the three-node laboratory; live host metrics are shown in the accompanying demo.
5	Dashboard displays at least three monitored nodes	Pass	The Grafana dashboard presented current host metrics through the private Tailscale control plane.
6	Detect sequence gaps	Pass	Receiver performs sequence number validation.
7	Append valid functional-test messages	Pass	receiver.cpp writes the seven-field CSV bridge; receiver.q parses, deduplicates, and inserts typed rows.
8	Refresh rolling best-bid/best-ask view	Pass	receiver.q executes on a one-second timer and derives recent best prices and associated quantities.
9	Rolling imbalance range	Pass by analysis	Five-second B/S quantity ratio is normalized to [-1, 1] for nonzero total quantity.
10	Source and receiver timestamps with percentile reporting	Pass	sender.cpp and receiver.cpp use CLOCK_REALTIME; receiver reports rolling P50/P95/P99/max.
11	Service persistence	Pass	Monitoring services configured to start after reboot.
12	FPGA insertion point and reference interface documented	Pass	Figure 1, source-verified event fields, q outputs, and the 33-test baseline define the first RTL migration boundary.

Table 4. Engineering requirements compliance matrix.

7. Challenges and Design Decisions

7.1 Dependency-Driven Scope and Development Order

The most important design decision was to build in dependency order. A hardware feed handler cannot be evaluated credibly without a controlled source, explicit framing, expected outputs, a host interface, timestamps, and observability. Each of those layers was therefore treated as an atomic deliverable. The resulting platform is modular: the sender, receiver, storage bridge, analytics, and monitoring stack have explicit responsibilities and can be changed independently. This sequencing positioned RTL as the next bounded engineering task rather than an isolated accelerator with no reliable system around it. Full Level 3 reconstruction, matching, strategy, and execution simulation were kept outside the scope because they are separate extensions rather than prerequisites for the first hardware comparison.

7.2 Hardware Compatibility

Consumer desktop hardware offers strong price to performance but does not provide the slot density or PCI Express topology of a server. The future accelerator, RTX 2070, and ConnectX card may require slot changes or removal of the GPU. The 1200 W power supply and large

chassis were selected to reduce electrical and mechanical constraints, but they do not create additional processor lanes. This tradeoff should be considered when selecting the eventual FPGA card.

7.3 Heterogeneous Service Management

The same monitoring architecture behaved differently on macOS, x86 Linux, and ARM Linux. Package paths, service managers, collectors, permissions, and network interface names varied. The Jetson collector issue and the laboratory routing issue required inspection of service status, endpoints, routes, and Prometheus target health. This debugging work was a major portion of the project even though it does not appear as application source code.

7.4 Evidence-Based Performance Claims

A 100 GbE-capable NIC does not prove 100 Gb/s application throughput, and an FPGA-ready chassis does not prove FPGA acceleration. The report therefore separates installed capability, source-verified functionality, measured results, and next-stage work. Similarly, the market-data pipeline is presented as a controlled research prototype rather than an institutional production platform. This evidence-based boundary strengthens the hardware roadmap because future RTL results can be compared against a clearly documented baseline.

8. Economic, Environmental, Ethical, and Safety Analysis

8.1 Economic Analysis

The priced component subtotal is $1,906 based on the amounts recorded in Appendix A. This subtotal includes the motherboard, cooling solution, and cabling amounts, but excludes replacement values for the reused RTX 2070, Mac mini, and physical switch. The project reduced cost by using a used enterprise Mellanox adapter and reusing existing equipment. The largest recorded single cost was the 32 GB DDR5 memory kit at $420. Appendix A distinguishes priced items from reused components and identifies the remaining model details that should be verified.

The system was not designed for commercial sale. Its economic value is educational and research oriented. A comparable cloud or hosted environment could avoid initial capital expense, but it would provide less control over physical networking, accelerator installation, and persistent hardware ownership. The modular laboratory can also be reused for operating systems, networking, GPU, embedded, and FPGA projects.

8.2 Environmental and Sustainability Considerations

The primary environmental impact is electricity use and embodied manufacturing cost. The 1200 W rating is a maximum capacity, not a claim of continuous consumption, but the workstation can still use substantially more power than the Jetson or Mac mini. The design reduces unnecessary operation by assigning persistent monitoring to the lower power Mac mini and allowing the main workstation to be shut down when experiments are not running. Reusing existing equipment and purchasing a used network adapter also extends component life.

The platform is maintainable because it uses standard replaceable desktop components, documented services, and open source monitoring tools. Future sustainability improvements include measuring wall power, defining automatic shutdown policies, consolidating idle services, and selecting an accelerator based on performance per watt rather than peak performance alone.

8.3 Ethical and Social Considerations

Low latency trading technology can improve liquidity and operational efficiency, but it can also increase unequal access when only well funded firms can purchase specialized data, colocation, and hardware. Systems can be misused for manipulative behavior or for deploying poorly tested strategies that create market risk. This project used synthetic data and did not connect to a brokerage or execute live orders. The design goal was education and measurement, not market manipulation or autonomous capital deployment.

A responsible extension should include risk limits, audit logs, kill switches, data licensing review, security controls, and clear separation between simulation and live execution. Results

should not be marketed as profitable strategy evidence unless transaction costs, data quality, execution assumptions, and out of sample validation are documented.

8.4 Health and Safety

The principal physical risks were electrical power, electrostatic discharge, sharp chassis edges, heat, fan obstruction, and heavy equipment handling. Components were installed with the power disconnected, and the selected case provides airflow and physical separation. High current power connectors and expansion cards must be seated correctly. Optical or high speed cable modules should be handled according to manufacturer instructions. Remote access also creates a cybersecurity responsibility, so management services were kept on a private overlay rather than intentionally exposed to the public Internet.

9. Teaming and Individual Contribution

This was an individual senior project. David El Kaim selected the architecture, sourced and assembled hardware, configured the operating systems and networks, deployed the observability stack, debugged service and routing issues, developed or coordinated the market data pipeline, documented the design, and evaluated the path toward FPGA acceleration. Dr. Maria Pantoja served as the faculty advisor. External software documentation and development assistants were used as references and productivity tools, but responsibility for validating the system and the claims in this report remains with the student.

10. Reflection

The project changed my understanding of what it means to build a low-latency system. At the beginning, I viewed the FPGA as the central deliverable. During implementation, I learned that the accelerator is one stage in a larger chain of dependencies. A hardware result cannot be interpreted if the feed is not reproducible, the network path is unclear, the storage layer loses context, or the surrounding machines cannot be observed when something fails.

The most valuable learning occurred in integration. I became more comfortable with Linux services, systemd, remote access, interface configuration, routing, Prometheus targets, Grafana dashboards, KDB+/Q, C++ socket behavior, and the differences between x86, ARM, and macOS environments. I also learned that hardware purchasing decisions have architectural consequences. PCI Express lanes, slot spacing, cooling, power, cable standards, firmware, and driver support can determine whether a theoretically compatible design is practical.

The main project-management lesson was the value of staged, evidence-based progress. Broad goals such as full Level 3 reconstruction, queue modeling, and FPGA acceleration become manageable only when they are decomposed into interfaces and independently testable steps. The final architecture now provides that decomposition. In future work, I would freeze the demonstration and test matrix earlier, preserve benchmark outputs as soon as they are generated, and treat documentation as part of implementation. The next step is now concrete: translate one verified software function into RTL and measure it against the existing reference path.

11. Conclusion and Future Work

This senior project designed and deployed a three-node, FPGA-ready market-data research testbed through a staged systems-engineering process. The completed work includes the custom B650 Linux workstation, Jetson Orin Nano receiving node, Mac mini monitoring node, physical laboratory network, Tailscale control plane, persistent Prometheus and Grafana observability, a configurable synthetic FIX-style generator, TCP stream framing, checksum and field validation, sequence monitoring, a seven-field CSV bridge, KDB+/Q ingestion logic, a five-second rolling best-bid/best-ask view, bid/ask quantity imbalance analytics, and sender-to-receiver percentile latency instrumentation. The C++17 verification suite completed 33 tests with no failures, and the implementation is preserved in a public repository and runbook.

RTL implementation was not completed during the documented period, and no hardware parser, synthesis result, timing-closure result, bitstream, or FPGA latency measurement is claimed. At project close, however, the system had reached a well-defined transition point: the message format, framing rules, reference outputs, test cases, host platform, monitoring environment, and comparison path were in place. The next active engineering phase is therefore a controlled migration of one bounded function into RTL, not a redesign of the overall system.

Recommended future work is prioritized as follows:

1. Freeze the first hardware interface and convert deterministic sender and unit-test cases into RTL input and expected-output vectors.
2. Implement and simulate a fixed-width parser or bid/ask imbalance module as the first bounded RTL stage.
3. Synthesize the module for a Zynq development board or Alveo card and report clock frequency, resource use, and timing slack.
4. Insert the FPGA stage into the existing feed path and compare its outputs and latency with the software baseline under identical replay input.
5. Archive a controlled benchmark containing synchronization status, message rate, duration, dropped messages, P50, P95, P99, and maximum latency.
6. Resolve and document the remaining laboratory-subnet routing path so monitoring and data traffic can be deliberately assigned to separate interfaces.
7. Extend from the rolling top-of-book view to multi-level depth only after hardware message correctness and recovery behavior are verified.

The final result is a complete systems and reference-model stage in a larger acceleration roadmap. It converts an abstract FPGA concept into a physical, observable, reproducible, and extensible laboratory. More importantly, it demonstrates the engineering progression required to connect hardware selection, networking, services, time-series storage, analytics, verification, and programmable-logic acceleration into one coherent architecture.

Bibliography

[1] Cal Poly Computer Engineering Program, “Senior Project Report Guidelines,” CPE Council, May 22, 2019.

[2] FIX Trading Community, “What is FIX?” and “FIX Protocol Online Specification.” Available: https://www.fixtrading.org/what-is-fix/ and https://www.fixtrading.org/online-specification/.

[3] KX, “Developing with kdb+ and the q language.” Available: https://code.kx.com/q/.

[4] Prometheus Authors, “Overview.” Available: https://prometheus.io/docs/introduction/overview/.

[5] Prometheus Authors, “Monitoring Linux host metrics with the Node Exporter.” Available: https://prometheus.io/docs/guides/node-exporter/.

[6] Grafana Labs, “About Grafana.” Available: https://grafana.com/docs/grafana/latest/introduction/.

[7] Tailscale Inc., “What is Tailscale?” Available: https://tailscale.com/docs/concepts/what-is-tailscale.

[8] NVIDIA, “ConnectX-4 Ethernet Adapter Cards User Manual.” Available: https://docs.nvidia.com/networking/display/ConnectX4EN.

[9] NVIDIA, “Jetson Orin Nano Developer Kit User Guide.” Available: https://docs.nvidia.com/jetson/orin-nano-devkit/user-guide/latest/.

[10] AMD, “Zynq 7000 SoCs.” Available: https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html.

[11] AMD, “Alveo U50 Data Center Accelerator Card.” Available: https://www.amd.com/en/products/accelerators/alveo/u50/a-u50-p00g-pq-g.html.

[12] AMD, “Vitis Unified Software Platform.” Available: https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis.html.

[13] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, “Limit order books,” Quantitative Finance, vol. 13, no. 11, pp. 1709-1742, 2013, doi: 10.1080/14697688.2013.803148.

[14] L. Harris, Trading and Exchanges: Market Microstructure for Practitioners. Oxford University Press, 2003.

[15] I. Aldridge, High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems, 2nd ed. Wiley, 2013.

[16] D. El Kaim, “KDB_Q_FIX_observability_pipeline,” GitHub repository. Available: https://github.com/DavidEK123/KDB_Q_FIX_observability_pipeline.

Appendix A. Bill of Materials

Subsystem	Component	Qty.	Cost	Status / Note
Primary workstation	AMD Ryzen 7 7700X processor	1	$260	Purchased / reported
Primary workstation	Corsair Vengeance DDR5 for AMD, 32 GB, 6400 MHz	1	$420	Purchased / reported; rated 6400 MT/s, 4800 MT/s observed at initial BIOS bring-up
Primary workstation	Samsung 990 Pro 2 TB NVMe SSD, up to 7450 MB/s	1	$200	Purchased / reported
Primary workstation	Corsair RM1200e power supply	1	$200	Purchased / reported
Primary workstation	NZXT H9 Flow case	1	$130	Purchased / reported
Primary workstation	Dell Mellanox ConnectX-4 CX455A 100GbE QSFP28/QSFP+ NIC, 06W1HY / JJN39	1	$76	Purchased / reported
Edge node	NVIDIA Jetson Orin Nano development kit	1	$250	Purchased / reported
Primary workstation	Gigabyte B650 AORUS ELITE AX motherboard	1	$250	Model confirmed by BIOS and assembly photographs
Primary workstation	NVIDIA RTX 2070 GPU	1	$0	Existing GPU; exact manufacturer/model not recorded
Primary workstation	NZXT Kraken Plus 240 liquid cooler	1	$70	Model confirmed from project photograph
Control node	2018 Intel Mac mini, 3.6 GHz quad core i3, 8 GB RAM, 128 GB SSD	1	$0	Existing monitoring/control server
Networking	Physical Ethernet switch	1	$0	Existing switch; exact model not recorded
Networking	Ethernet, DAC, QSFP, optics, and adapters	Var.	$50	Recorded allowance; exact types not recorded
Priced subtotal	Recorded-price components		$1,906	Excludes reused GPU, Mac mini, and switch

Table A1. Project bill of materials. Priced component subtotal: $1,906, excluding reused items without assigned replacement values.

Priced component subtotal: $1,906. This amount includes every component with a recorded project price in Table A1 and excludes replacement values for reused equipment. The GPU, Mac mini, and physical switch are listed at $0 project cost because they were reused rather than purchased for this build.

Appendix B. Analysis of Senior Project Design

B.1 Summary of Functional Requirements

The system provides a three-node research environment for generating, transporting, receiving, storing, analyzing, and monitoring timestamped market-data events. It provides centralized infrastructure monitoring, remote administration, rolling top-of-book analytics, a

source-verified software reference path, and a documented FPGA insertion boundary for the next implementation phase.

B.2 Primary Constraints

The primary constraints were budget, time, PCI Express slot and lane availability, high-speed networking compatibility, heterogeneous operating systems, limited access to enterprise hardware, and the dependency chain that had to be completed before an RTL result could be evaluated in an end-to-end environment.

B.3 Economic

- Original estimated component cost: not formally recorded.
- Known priced component subtotal: $1,906, excluding reused equipment without assigned replacement values.
- Actual final cash expenditure was not independently reconstructed from receipts. The priced bill of materials totals $1,906, excluding replacement values for reused equipment.
- Additional development equipment: personal laptop, monitors, ordinary Ethernet equipment, and existing computers.
- Original and actual development time: to be estimated from project calendar before submission.
- Commercial production: not applicable. The project is a reusable research testbed rather than a manufactured product.

B.4 Environmental

Environmental impacts include component manufacturing, electricity use, and electronic waste. Reuse of the Mac mini, GPU, and used ConnectX adapter reduced new purchases. Separating persistent monitoring onto a lower power machine allows the high power workstation to remain off when not required.

B.5 Manufacturability

The project is assembled from commercially available modular components and does not require custom manufacturing. Reproduction depends on component compatibility, network adapters, cabling, driver availability, and documented configuration. A standardized deployment script would improve reproducibility.

B.6 Sustainability

The modular system can be repaired and upgraded one component at a time. Open source software and standard network protocols reduce vendor lock in. Sustainability improvements

include power measurement, idle shutdown, configuration management, and selecting future accelerators according to performance per watt.

B.7 Ethical

The platform can be used to study systems that affect financial markets. Ethical concerns include unequal access to speed, misuse for manipulative trading, poor risk controls, and exaggerated performance claims. The current system uses synthetic data and no live orders. Future live use would require compliance review, safeguards, and auditability.

B.8 Health and Safety

Health and safety concerns include electrical power, ESD, heat, airflow, sharp chassis edges, and heavy equipment handling. Cybersecurity is also relevant because remote access services can expose infrastructure when misconfigured. Private overlay access and standard hardware safety practices reduce these risks.

B.9 Social and Political

Low latency systems raise questions about fairness, access, market stability, and the concentration of technological advantages. The project is educational and does not advocate unrestricted deployment. Its observability and replay features can support safer testing and clearer understanding of system behavior.

B.10 Development

New tools and techniques learned independently included KDB+/Q, Prometheus, Grafana, Node Exporter, Tailscale, Linux systemd services, multi-interface routing, ARM versus x86 package deployment, C++ socket framing and validation, enterprise QSFP network-adapter integration, automated protocol testing, and modular software-to-RTL migration planning for PCI Express FPGA accelerators.